

Documentation Technique — Backend

Projet : Night Sky Viewer
Stack : Python 3.12 · FastAPI 0.129 · SQLAlchemy 2.0 · SQLite · AstroPy 7.2 · NumPy 2.4
Version : 0.1.0

Table des matières

- 1. [Architecture générale](#)
- 2. [Structure des fichiers](#)
- 3. [Modèles de données](#)
- 4. [API Reference](#)
- 5. [Services métier](#)
- 6. [Système de cache](#)
- 7. [Configuration](#)
- 8. [Base de données](#)
- 9. [Dépendances](#)

1. Architecture générale

Le backend suit un pattern **4 couches** strict qui garantit une séparation des responsabilités claire.



Couche	Dossier	Peut appeler	Interdit
Router	app/api/	Service, Schéma	Repository, BD
Service	app/services/	Repository	BD directement
Repository	app/repositories/	BD (SQLAlchemy)	—
Schema	app/schemas/	—	—

Model	app/models/	—	—
--------------	-------------	---	---

2. Structure des fichiers

```

backend/
├─ app/
│   ├── main.py                # Point d'entrée FastAPI, CORS, lifespan
│   ├── config.py              # Settings (pydantic-settings, .env)
│   ├── database.py            # Engine SQLAlchemy, session factory, Base
│   └─ api/
│       ├── stars.py           # GET /api/stars/*
│       ├── constellations.py  # GET /api/constellations/*
│       ├── observation_points.py # GET /api/observation-points/*
│       ├── iss.py             # GET /api/iss/tle
│       ├── rovers.py          # GET /api/rovers/positions
│       └─ satellites.py        # GET /api/satellites/tle, GET /api/satellites/groups
│   └─ models/
│       ├── star.py            # ORM Star
│       ├── constellation.py    # ORM Constellation + ConstellationStar
│       ├── observation_point.py # ORM ObservationPoint
│       └─ rover.py            # ORM Rover
│   └─ schemas/
│       ├── star.py            # StarDetail, VisibleStarResponse
│       └─ constellation.py    # ConstellationListResponse,
│                               ConstellationDetailResponse, BestLocationResponse
│       ├── observation_point.py # ObservationPointResponse
│       ├── rovers.py           # RoverPosition, RoverPositionsResponse
│       └─ satellite.py         # SatelliteTLE, SatelliteTLEResponse
│   └─ repositories/
│       ├── star_repository.py
│       ├── constellation_repository.py
│       ├── observation_point_repository.py
│       └─ rover_repository.py
│   └─ services/
│       ├── astronomy_service.py # Calculs AstroPy vectorisés
│       ├── cache_service.py     # Cache LRU + TTL
│       ├── rovers_service.py    # RoversService + seed defaults
│       └─ satellite_service.py  # Proxy CelesTrak + cache TTL 12h par groupe
├─ data/
│   └─ nightsky.db              # Base SQLite (générée au démarrage)
├─ scripts/
│   └─ import_stars.py          # Import catalogue HYG
├─ Dockerfile
├─ requirements.txt
└─ .env

```

3. Modèles de données

3.1 Star

Table : stars

Colonne	Type	Description
id	INTEGER PK	Identifiant interne
hip_id	INTEGER	ID Hipparcos (nullable)
hd_id	INTEGER	ID Henry Draper (nullable)
proper_name	TEXT	Nom commun (ex : Sirius, Vega)
ra	REAL	Ascension droite J2000 (degrés)
dec	REAL	Déclinaison J2000 (degrés)
magnitude	REAL	Magnitude apparente (indexée)
abs_magnitude	REAL	Magnitude absolue
spectral_type	TEXT	Type spectral (ex : G2V)
color_index	REAL	Indice de couleur B-V
distance_ly	REAL	Distance en années-lumière
constellation_abbr	TEXT(3)	Abréviation IAU (ex : ORI)

Index composite : (magnitude, dec) — accélère le pré-filtrage avant le calcul AstroPy.

Relation : many-to-many avec Constellation via constellation_stars .

Source : Catalogue HYG Database v3.7.

3.2 Constellation

Table : constellations

Colonne	Type	Description
id	INTEGER PK	—
name	TEXT UNIQUE	Nom anglais (ex : Orion)
abbreviation	TEXT(3) UNIQUE	Abréviation IAU
name_fr	TEXT	Nom en français
center_ra	REAL	RA du centre (degrés)
center_dec	REAL	Dec du centre (degrés)
lines_data	TEXT (JSON)	Pattern lignes : [[hip1, hip2], ...]
description	TEXT	Description libre

3.3 ConstellationStar (table d'association)

Table : `constellation_stars`

Colonne	Type	Description
<code>id</code>	INTEGER PK	—
<code>constellation_id</code>	FK → <code>constellations.id</code>	—
<code>star_id</code>	FK → <code>stars.id</code>	—

3.4 ObservationPoint

Table : `observation_point`

Représente une ville/lieu d'observation avec ses coordonnées géographiques.

3.5 Rover (NEW)

Table : `rovers`

Position et métadonnées d'un rover martien. Positions éditables en base sans redéploiement.

Colonne	Type	Description
<code>id</code>	INT PK	—
<code>slug</code>	TEXT UNIQ	Identifiant unique (ex: curiosity)
<code>name</code>	TEXT	Nom affiché
<code>agency</code>	TEXT	Agence spatiale (ex: NASA / MSL)
<code>active</code>	BOOLEAN	Mission en cours
<code>latitude</code>	FLOAT	Latitude Mars (degrés, N positif)
<code>longitude</code>	FLOAT	Longitude Mars (degrés, E positif)
<code>landing_site</code>	TEXT	Nom du site d'atterrissage

Rovers par défaut : Curiosity, Perseverance, Opportunity, Spirit, Zhurong.

4. API Reference

Base URL

`http://localhost:8000`

La documentation interactive Swagger est disponible à `http://localhost:8000/docs` .

4.1 Étoiles — `/api/stars`

GET /api/stars/visible

Retourne toutes les étoiles visibles à l'œil nu depuis un lieu et un instant donnés.

Paramètres de requête :

Paramètre	Type	Requis	Default	Description
lat	float	✓	—	Latitude (−90 à +90)
lon	float	✓	—	Longitude (−180 à +180)
timestamp	datetime ISO 8601	✗	maintenant	Temps d'observation UTC
mag_limit	float	✗	6.0	Magnitude limite (−2 à 12)

Algorithme :

1. Pré-filtrage SQL : `magnitude <= mag_limit AND dec BETWEEN lat-90 AND lat+90`
2. Transformation vectorisée RA/Dec → Az/Alt via AstroPy (un seul `transform_to()`)
3. Filtre : `altitude > 0°`
4. Cache 10 minutes (clé arrondie à 5 minutes)

Réponse exemple :

```
[
  {
    "id": 32349,
    "proper_name": "Sirius",
    "hip_id": 32349,
    "magnitude": -1.46,
    "spectral_type": "A1V",
    "constellation_abbr": "CMA",
    "distance_ly": 8.6,
    "azimuth": 178.4321,
    "altitude": 42.1034
  }
]
```

GET /api/stars/{star_id}

Retourne les détails complets d'une étoile par son ID interne.

Erreurs : 404 si l'étoile n'existe pas.

GET /api/stars/search/{query}

Recherche d'étoiles par nom propre.

Paramètres :

Paramètre	Type	Description
query	string (≥2 chars)	Terme de recherche

limit	int (1-100)	Nombre de résultats (défaut: 20)
-------	-------------	----------------------------------

4.2 Constellations — /api/constellations

GET /api/constellations

Retourne les 88 constellations IAU. **Résultat mis en cache statique (TTL 24h).**

Réponse : liste de `ConstellationListResponse` incluant `lines_data` (JSON) et `pattern_stars` (coordonnées RA/Dec des étoiles du dessin).

GET /api/constellations/search?q=...

Recherche par nom (français ou anglais), minimum 2 caractères.

GET /api/constellations/{constellation_id}

Détails complets d'une constellation, incluant la liste des IDs d'étoiles du pattern.

Erreurs : 404 si non trouvée.

GET /api/constellations/{constellation_id}/best-location

Trouve le meilleur point d'observation parmi tous les `ObservationPoint` enregistrés pour voir une constellation à un instant donné.

Algorithme :

1. Récupère tous les points d'observation
2. Calcule l'altitude de la constellation depuis chaque point (appel AstroPy vectorisé)
3. Retourne le point avec l'altitude maximale
4. Score de visibilité = $(\text{altitude} / 90^\circ) \times 100$

Paramètres :

Paramètre	Type	Description
timestamp	datetime	Optionnel, défaut: maintenant

Réponse :

```
{
  "observation_point_id": 12,
  "observation_point_name": "Paris",
  "latitude": 48.85,
  "longitude": 2.35,
  "constellation_altitude": 64.23,
  "visibility_score": 71.4
}
```

Erreurs : 404 si aucun point ne permet de voir la constellation, 422 si coordonnées du centre manquantes.

4.3 Points d'observation — /api/observation-points

GET /api/observation-points

Liste tous les points d'observation disponibles.

GET /api/observation-points/{point_id}

Détails d'un point d'observation. Erreur 404 si inexistant.

4.4 ISS — /api/iss

GET /api/iss/tle

Proxy vers Celestrak pour les données TLE de l'ISS. **Résultat mis en cache 12 heures** (évite le rate-limiting).

Sources testées (fallback automatique) :

1. <https://celestrak.org/NORAD/elements/stations.txt>
2. <https://live.ariss.org/iss.txt>

L'ISS est identifiée par son nom ISS (ZARYA) ou son NORAD ID 25544 .

Réponse :

```
{
  "name": "ISS (ZARYA)",
  "line1": "1 25544U 98067A 24001.50000000 .00001234 00000-0 12345-4 0 9999",
  "line2": "2 25544 51.6400 123.4567 0001234 56.7890 303.2109 15.49876543123456",
  "timestamp": 12345.678
}
```

Erreurs : 502 si toutes les sources sont inaccessibles.

4.5 Rovers — /api/rovers (NEW)

GET /api/rovers/positions

Retourne les positions de tous les rovers martiens.

Algorithme :

1. Récupère tous les rovers de la table rovers
2. Si la table est vide, insère les positions par défaut (seed) automatiquement
3. Retourne la liste avec métadonnées statiques mergées côté frontend

Réponse :

```
{
  "rovers": [
    {
      "slug": "curiosity",
      "name": "Curiosity",
      "agency": "NASA / MSL",
      "active": true,
    }
  ]
}
```

```
    "latitude": -4.6,
    "longitude": 137.4,
    "landing_site": "Gale Crater"
  }
],
"total": 5
}
```

4.6 Satellites — /api/satellites

GET /api/satellites/tle?group={group}

Proxy vers CelesTrak pour les TLE d'un groupe de satellites. **Résultat mis en cache 12 heures par groupe.**

Paramètres de requête :

Paramètre	Type	Requis	Default	Description
group	string	✗	"stations"	Groupe CelesTrak (voir liste ci-dessous)

Groupes disponibles : stations , starlink , gps-ops , weather , resource , science , galileo , active

Réponse :

```
{
  "group": "stations",
  "count": 42,
  "satellites": [
    {
      "name": "ISS (ZARYA)",
      "line1": "1 25544U 98067A ...",
      "line2": "2 25544 51.6400 ..."
    }
  ]
}
```

Erreurs : 400 si le groupe est inconnu, 502 si CelesTrak est inaccessible ou retourne une réponse de rate-limit (1 fetch/2h par IP).

GET /api/satellites/groups

Liste tous les groupes de satellites disponibles.

Réponse : {"groups": ["active", "galileo", "gps-ops", ...]}

4.7 Health — /

GET /

Ping rapide : retourne nom, version et "status": "running" .

GET /api/health

Health check détaillé : vérifie la connexion BD et retourne le nombre d'étoiles chargées.

5. Services métier

5.1 AstronomyService

backend/app/services/astronomy_service.py

Toutes les méthodes sont **statiques** et utilisent la **vectorisation NumPy/AstroPy** (×50 plus rapide qu'une boucle Python).

`compute_visible_stars(stars, latitude, longitude, timestamp)`

Transforme les coordonnées équatoriales (RA/Dec, J2000, ICRS) en coordonnées horizontales (Azimut/Altitude) pour un observateur donné.

Pipeline :

```
# 1. Extraction vectorisée
ra_array = np.array([s.ra for s in stars])
dec_array = np.array([s.dec for s in stars])

# 2. Référentiel observateur
location = EarthLocation(lat=..., lon=...)
altaz_frame = AltAz(obstime=Time(timestamp), location=location)

# 3. Transformation TOUTES étoiles en 1 appel
sky_coords = SkyCoord(ra=ra_array*u.degree, dec=dec_array*u.degree, frame="icrs")
altaz_coords = sky_coords.transform_to(altaz_frame)

# 4. Filtre altitude > 0°
visible_mask = altaz_coords.alt.degree > 0
```

`compute_best_observation_point(ra, dec, points, timestamp)`

Calcule l'altitude d'un objet céleste depuis **tous les points d'observation simultanément** grâce à `EarthLocation` vectorisé, puis retourne le meilleur via `np.argmax`.

5.2 CacheService

backend/app/services/cache_service.py

Deux niveaux de cache en mémoire (bibliothèque `cachetools`) :

Niveau	TTL	Usage
Dynamique (TTLCache)	10 minutes	Positions d'étoiles calculées
Statique (TTLCache)	24 heures	Constellations, données immuables

La clé de cache pour les positions est arrondie à **5 minutes** (granularité) pour maximiser les hits.

5.3 RoversService

backend/app/services/rovers_service.py

Gère les positions des rovers martiens. Seed la table automatiquement si vide.

Méthodes :

- `get_all_positions(db) → list[dict]` — retourne tous les rovers + seed si nécessaire
- `_seed_defaults(db)` — insère les 5 rovers par défaut

5.4 SatelliteService

backend/app/services/satellite_service.py

Proxy stateless vers Celestrak — pas de modèle SQLAlchemy (données non persistées).

Fonctionnement :

- Cache en mémoire `TTLCache(maxsize=10, ttl=43200)` — 12h par groupe
- Parse le format TLE 3-lignes (nom, line1, line2) avec validation stricte (préfixe `"1 "` / `"2 "` + longueur ≥ 69 caractères)
- Détecte la réponse de rate-limit Celestrak ("has not updated") et lève `RuntimeError`

Constantes :

- `VALID_GROUPS` — ensemble des 8 groupes supportés
- `CELESTRAK_GROUPS` — mapping groupe → URL Celestrak

Méthodes :

- `fetch_satellite_tles(group) → list[SatelliteTLE]` — async, lève `ValueError` si groupe inconnu
- `parse_tle_text(text) → list[SatelliteTLE]` — parse un bloc texte TLE en liste de schémas

6. Système de cache

Endpoint	Cache	TTL
GET /api/stars/visible	Oui (dynamique)	10 min
GET /api/constellations	Oui (statique)	24 h
GET /api/iss/tle	Oui (TTLCache)	12 h
GET /api/satellites/tle	Oui (TTLCache)	12 h (par groupe)
Tous les autres	Non	—

7. Configuration

Fichier `.env` (racine `backend/`) :

```
DATABASE_URL=sqlite:///./data/nightsky.db
APP_NAME=Night Sky Viewer
```

```
APP_VERSION=0.1.0
ENVIRONMENT=development
DEBUG=true
CORS_ORIGINS=["http://localhost:5173","http://localhost:4173","http://localhost:3000"]
CACHE_TTL_STARS=600
CACHE_TTL_STATIC=86400
CACHE_MAX_SIZE=512
DEFAULT_MAG_LIMIT=6.0
MAX_STARS_RESPONSE=10000
```

Variables chargées via `pydantic-settings` (classe `Settings`).

8. Base de données

- **Moteur** : SQLite 3 (fichier `backend/data/nightsky.db`)
- **ORM** : SQLAlchemy 2.0
- **Initialisation** : `create_tables()` appelé au démarrage via le hook `lifespan` FastAPI
- **Import catalogue** : script `backend/scripts/import_stars.py` — lit le CSV HYG v3.7 via `pandas` et insère les étoiles en batch

Schéma relationnel

```
stars ————— constellation_stars ——— constellations
id PK                      star_id FK          id PK
hip_id                    constellation_id FK    name
...                               abbreviation
                               lines_data (JSON)

observation_point
id PK
name
latitude
longitude
```

9. Dépendances

Paquet	Version	Rôle
<code>fastapi</code>	0.129.0	Framework web ASGI
<code>uvicorn[standard]</code>	0.41.0	Serveur ASGI
<code>sqlalchemy</code>	2.0.46	ORM
<code>pydantic</code>	2.12.5	Validation des schémas
<code>pydantic-settings</code>	2.13.1	Gestion de la configuration
<code>astropy</code>	7.2.0	Calculs astronomiques (ICRS → AltAz)
<code>numpy</code>	2.4.2	Vectorisation des calculs

pandas	3.0.1	Import du catalogue HYG
cachetools	5.5.1	Cache LRU/TTL en mémoire
httpx	0.28.1	Client HTTP async (proxy ISS)
pytest	8.3.4	Tests unitaires
pytest-asyncio	0.25.2	Tests endpoints async